

Week 6 Assignment

Q1: Explain the roles of the Driver, Cluster Manager, and Executor in a Spark application.

Answer:

Apache Spark follows a **Master-Worker architecture**. The three main components are:

1. Driver Program

- The Driver is the main program of a Spark application.
- It creates the SparkSession.
- It converts user code into tasks.
- It builds the **DAG (Directed Acyclic Graph)**.
- It sends tasks to the executors and collects the final results.

Example: When we run a PySpark program in Google Colab, the Driver creates the execution plan for our DataFrame operations.

2. Cluster Manager

- The Cluster Manager manages the cluster resources.
- It allocates CPU and memory to executors.
- It starts and monitors executors.

Examples of Cluster Managers:

- Standalone
- Hadoop YARN
- Kubernetes
- Apache Mesos

3. Executors

- Executors are worker processes.
- They perform the actual computations.
- They store intermediate data in memory.
- They return the processed results to the Driver.

Workflow

User Program

|



Driver Program

|



Cluster Manager

|



Executors

|



Result

Q2: How does Spark's Lazy Evaluation strategy improve performance when chain-processing large datasets?

Answer:

Lazy Evaluation means Spark does not execute transformations immediately. Instead, Spark records all transformations and creates a DAG (Directed Acyclic Graph).

Execution starts only when an **Action** such as `show()`, `count()`, or `collect()` is called.

Advantages

- Combines multiple transformations into a single execution plan.
- Eliminates unnecessary computations.
- Reduces disk I/O.
- Optimizes resource usage.
- Improves execution speed.

Example

```
filtered_df = df.filter(col("price") > 100)
selected_df = filtered_df.select("product_id", "price")
```

No execution happens here.

```
selected_df.show()
```

Execution starts only here.

Q3: Write a Spark command to read a CSV file located at "data/source.csv", ensuring the first row is treated as a header and inferSchema is enabled.

Answer

```
df = spark.read.csv(
    "data/source.csv",
    header=True,
    inferSchema=True
)
```

Explanation

- header=True → Uses the first row as column names.
 - inferSchema=True → Automatically detects the correct data types.
-

Q4: What is the difference between CSV and Parquet in terms of storage (row-based vs. columnar) and why does it matter for performance?

Answer

CSV	Parquet
Row-based storage	Column-based storage
Larger file size	Smaller file size
No compression by default	Highly compressed
Slower reading	Faster reading

CSV

Parquet

Suitable for data exchange Suitable for analytics

Why does it matter?

Parquet reads only the required columns, reducing disk I/O and memory usage.

For example, if we only need the price column, Parquet reads only that column, whereas CSV reads the entire row.

Therefore, Parquet provides faster query execution and better performance.

Q5: Given a DataFrame df, write a query to select the columns product_id and price where the category is 'Electronics'.

Answer

```
df.filter(col("category") == "Electronics") \
    .select("product_id", "price") \
    .show()
```

Q6: Write the code to revise a DataFrame by renaming the column old_name to new_name and casting the price column from a String to a Double.

Answer

```
revised_df = df.withColumnRenamed(
    "old_name",
    "new_name"
)
revised_df = revised_df.withColumn(
    "price",
    col("price").cast("Double")
)

revised_df.printSchema()
```

Q7: How does Spark use the Lineage Graph (DAG) to provide fault tolerance if a worker node fails?

Answer

Spark maintains a **Lineage Graph (DAG)** that records every transformation applied to the data.

If an executor fails, Spark does not need to reload the entire dataset. Instead, it recomputes only the lost partition using the transformations stored in the DAG.

Benefits

- Automatic recovery
- No manual backup required
- Efficient fault tolerance
- Faster recovery

Q8: Write a query to filter a DataFrame df_orders for rows where the status is 'Completed' AND the amount is greater than 1000.

Answer

```
df_orders.filter(  
    (col("status") == "Completed") &  
    (col("amount") > 1000)  
).show()
```

Q9: Explain the concept of Predicate Pushdown in Parquet and how it affects the amount of data loaded into memory.

Answer

Predicate Pushdown is an optimization technique used by Spark while reading Parquet files.

Instead of reading the complete dataset first and then applying filters, Spark pushes the filter conditions to the Parquet file reader.

Benefits

- Reads fewer rows.

- Reduces memory usage.
- Reduces disk I/O.
- Improves query performance.

Example:

```
df.filter(col("age") > 25)
```

Spark loads only the required records instead of the entire dataset.

Q10: Write a code snippet to add a new column `final_price` which is the `base_price` multiplied by 1.18 (18% tax).

Answer

```
df = df.withColumn(  
    "final_price",  
    col("base_price") * 1.18  
)
```

```
df.show()
```

Q11: What is the difference between Transformations and Actions? Provide two examples of each.

Answer

Transformations

Actions

Create a new DataFrame Trigger execution

Lazy operations

Execute immediately

Transformation Examples

```
filter()
```

```
select()
```

Action Examples

show()

count()

Explanation

Transformations build the execution plan but do not execute it.

Actions trigger Spark to execute all pending transformations.

Q12: Write the Spark command to load a Parquet file from "path/to/input", filter out any rows where user_id is null, and save the result as a CSV at "path/to/output".

Answer

```
spark.read.parquet("path/to/input") \
    .filter(col("user_id").isNotNull()) \
    .write \
    .mode("overwrite") \
    .option("header", True) \
    .csv("path/to/output")
```

Q13: In Spark Architecture, what is the difference between Client Mode and Cluster Mode?

Answer

Client Mode

Driver runs on the local machine

Suitable for development and testing

If the client disconnects, the application may stop

Easier to debug

Cluster Mode

Driver runs inside the cluster

Suitable for production

Continues running even if the client disconnects

Better resource management

Q14: Write a query to filter a dataset for rows where the region is 'North' OR the priority is 'High'.

Answer

```
df.filter(  
    (col("region") == "North") |  
    (col("priority") == "High")  
).show()
```

Q15: When exploring a dataset, why is it safer to use `.show(5)` instead of `.collect()` on a multi-terabyte dataset?

Answer

The `show(5)` function displays only the first five rows of the DataFrame, whereas `collect()` retrieves all the data from the executors and loads it into the Driver's memory.

For a multi-terabyte dataset, using `collect()` can cause the Driver to run out of memory, resulting in poor performance or application failure.

Advantages of `show(5)`

- Displays only a few records.
- Uses very little memory.
- Faster execution.
- Safe for large datasets.

Disadvantages of `collect()`

- Loads the entire dataset into Driver memory.
- High memory consumption.
- Can cause `OutOfMemory` errors.
- Not recommended for large datasets.

Therefore, `show(5)` is the preferred method for exploring large datasets because it is memory-efficient and much safer than `collect()`.